Laravel Interview Questions

1. Core Laravel Concepts

1. What is Laravel, and why is it popular?
2. Explain Laravel's **MVC architecture**.

Model: The Model represents the application's data and business logic.
View: The View is responsible for presenting data to the user.
Controller: The Controller acts as an intermediary between the Model and the View.

3. What is the role of **Service Providers** in Laravel?

In Laravel, a **Service Provider** is a central place to register and bootstrap application services, bindings, and configuration into the service container, ensuring they are loaded and ready before handling any request.

Laravel me **Service Provider** ek aisi jagah hoti hai jaha hum apne application ke **services, classes, aur configuration ko register aur bootstrap** karte hain, taaki request aane se pehle hi sab ready ho jaye.

Simple shabdon me:
Service Provider Laravel ko batata hai ki kaun-kaun se features ya services load karni hain aur kaise load karni hain.

Example kaam:

- Database connection set karna
- Custom class ko bind karna
- Event listener register karna
- View me common data share karna

4. What are **Facades** in Laravel, and how do they work?

In Laravel, **Facades** provide a static interface to classes in the Service Container, allowing you to call their methods without creating instances manually.

Laravel me **Facades** ek shortcut hai jo Service Container me stored classes ko static method ki tarah use karne deta hai.

5. Explain the concept of **Service Container** in Laravel.

It is a important Powerful tool for Resolving Class Dependencies and performing dependency injection in Laravel

6. What are **Contracts** in Laravel?

In Laravel, **Contracts** are interfaces that define which methods a service should have, so the code remains flexible and maintainable

Laravel me **Contracts** aise interfaces hote hain jo batate hain ki kisi service me kaunse methods hone chahiye, taaki code flexible aur maintainable rahe.

7. Difference between **Contracts** and **Facades**.
8. Explain **Laravel's request lifecycle**.

Browser Request
↓
public/index.php
↓
HTTP Kernel (Middleware)
↓
Service Providers Bootstrapped
↓
Route Matched
↓
Controller / Closure
↓
Model / Database (if needed)
↓
View or JSON Response
↓
HTTP Kernel terminate()
↓
Browser Response

9. What is **Artisan** in Laravel? Give some useful Artisan commands.
10. Difference between `require` and `require_once` in PHP and how it's relevant to Laravel autoloading.

2. Database & Eloquent

1. How does **Eloquent ORM** work in Laravel?

Eloquent ORM Laravel ka built-in Object-Relational Mapper hai jo database tables ko models ke form me represent karta hai aur database ke saath object-oriented tarike se interact karne ki facility deta hai bina raw SQL queries likhe.

Standard Definition (2–3 Lines)

"Eloquent ORM ek Object-Relational Mapping system hai jo Laravel framework me built-in hai. Ye har database table ke liye ek model provide karta hai jisse hum CRUD operations aur relationships ko easily manage kar sakte hain, bina direct SQL queries likhe."

2. Difference between `get()` and `first()` in Eloquent.

Aspect	`get()`	`first()`
Return Type	Collection of models	Single model instance (or `null`)
Records Fetched	All matching records	Only the first matching record
Usage	When you need multiple rows	When you need just one row
Empty Result	Returns empty collection `[]`	Returns `null`

Aspect	`find()`	`first()`
Based On	Primary key (`id` by default)	Query conditions
Multiple Results	Can accept an array of IDs (returns collection)	Always returns only the first matching record
Usage	When you know the record's **primary key**	When you want the **first record** matching some condition
Return Type	Model instance, Collection (if multiple IDs), or `null`	Model instance or `null`

3. Explain **Eager Loading** vs **Lazy Loading** in Eloquent.

Lazy Loading - Lazy Loading **loads relationships only when accessed (can cause N+1 queries).(Relationships ka data tabhi load hoga jab aap usko accessed karoge)**

```
$users = User::all();
```

```
foreach ($users as $user) {
   echo $user->profile->bio; // Runs another query for each user
}
```

Eager Loading- Relationships are **preloaded in advance** with the main query using `with()`.
(Relationships ka data phale se hi shath me load kar liya jaata hai)

```
$users = User::with('profile')->get();
foreach ($users as $user) {
   echo $user->profile->bio; // No extra queries, data already loaded
}
```

4. How do you perform **soft deletes** in Laravel?

In Laravel, **Soft Delete** means marking a record with a `deleted_at` timestamp instead of permanently removing it, so it can be restored later.

5. What is **Query Scope** in Laravel?

In Laravel, **Query Scope** means defining reusable query conditions inside a model for easy reuse.

6. Difference between `pluck()` and `select()`.

`pluck()` gets values of a single column, while `select()` chooses which columns to fetch in query results.

7. How do you perform database **transactions** in Laravel?

A **transaction** ensures that a group of database operations either all succeed or all fail, maintaining data integrity.

1. Ways to Perform Transactions

Using `DB::transaction()` (Recommended)

```
use Illuminate\Support\Facades\DB;

DB::transaction(function () {
      // All queries inside here are atomic
      User::create(['name' => 'John']);
      Profile::create(['user_id' => 1]);
});
```

2. **Manual Begin, Commit, Rollback**

```
DB::beginTransaction();

try {
    User::create(['name' => 'John']);
    Profile::create(['user_id' => 1]);

    DB::commit(); // Save changes
} catch (\Exception $e) {
    DB::rollBack(); // Undo changes
    throw $e;
}
```

8. What is the use of **migrations** in Laravel?
9. How do you create and rollback migrations?

1. Create a Migration- php artisan make:migration create_users_table
2. Run Migrations - php artisan migrate
3. Rollback Migrations - php artisan migrate:rollback

Rollback **all migrations**: php artisan migrate:reset
Rollback & re-run migrations: php artisan migrate:refresh
Fresh migration (drop all tables & re-run): php artisan migrate:fresh

10. How do you perform **many-to-many** relationships in Laravel? Give an example.

A **many-to-many relationship** occurs when multiple records in one table are related to multiple records in another table.
Example: A `User` can have many `Roles`, and each `Role` can belong to many `Users`.

3. Routing & Middleware

1. Difference between **web.php** and **api.php** routes.

`web.php` is for browser-based routes with session/CSRF (stateful), while `api.php` is for REST APIs with JSON responses (stateless).

Aspect	web.php	api.php
Purpose	Used for web routes (browser-based pages, forms, views).	Used for API routes (RESTful APIs, JSON responses).
Middleware	Routes automatically get the `web`	Routes automatically get the `api`

Aspect	web.php	api.php
	middleware (sessions, CSRF, cookies).	middleware (stateless, no sessions/CSRF).
State	Stateful → Maintains session, authentication via cookies.	Stateless → No sessions, each request must carry authentication (like tokens).
Response Type	Usually returns HTML views.	Usually returns JSON data.
Example	`php Route::get('/home', [HomeController::class, 'index']);`	`php Route::get('/users', [UserController::class, 'index']);`

2. How do you define a **route with parameters** in Laravel?

```
Route::get('/user/{id}', function ($id) {
   return "User ID is " . $id;
});
```

3. What is **Route Model Binding**? Difference between **Implicit** and **Explicit** binding.
4. How do you create and use **middleware** in Laravel?

1. Create Middleware- php artisan make:middleware CheckAge
2. Define Logic in Middleware-

```
namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;

class CheckAge
{
   public function handle(Request $request, Closure $next)
   {
      if ($request->age < 18) {
         return redirect('home'); // block if age < 18
      }
      return $next($request); // allow request
   }
}
```

Register Middleware - app/Http/Kernel.php

```
protected $routeMiddleware = [
    'checkAge' => \App\Http\Middleware\CheckAge::class,
];
```

Use Middleware in Routes –

```
Route::get('/restricted', function () {
    return "You are old enough!";
})->middleware('checkAge');
```

5. How to pass data from middleware to the view?

```
use Illuminate\Support\Facades\View;

class ShareDataMiddleware
{
    public function handle($request, Closure $next)
    {
        View::share('role', 'Admin');
        return $next($request);
    }
}
```

Now in any Blade file:

```
<p>User Role: {{ $role }}</p>
```

6. How do you restrict access to certain routes based on user roles?

   The most common way is to create a **custom middleware** that checks the logged-in user's role before allowing access.

   1. Create Middleware - php artisan make:middleware RoleMiddleware
   2. Define Logic (`app/Http/Middleware/RoleMiddleware.php`)

```
namespace App\Http\Middleware;

use Closure;
use Illuminate\Support\Facades\Auth;
```

```
class RoleMiddleware
{
    public function handle($request, Closure $next, $role)
    {
        if (!Auth::check() || Auth::user()->role !== $role) {
            abort(403, 'Unauthorized'); // Or redirect()->route('home')
        }
        return $next($request);
    }
}
```

3. Register Middleware (`app/Http/Kernel.php`)

```
protected $routeMiddleware = [
    'role' => \App\Http\Middleware\RoleMiddleware::class,
];
```

4. Apply Middleware to Routes (`web.php`)

```
Route::get('/admin/dashboard', function () {
    return "Welcome Admin!";
})->middleware('role:admin');

Route::get('/user/dashboard', function () {
    return "Welcome User!";
})->middleware('role:user');
```

7. What are **named routes** and their advantages?

A **named route** is a route with a specific name that you can use to generate URLs or redirects, instead of hardcoding paths.

Advantages of Named Routes – Flexibility, Readability, Maintainability, Parameter Handling, Consistency

Blade & Frontend

1. What is **Blade templating** in Laravel?

Blade is **Laravel's built-in templating engine** that allows you to write clean, reusable, and dynamic HTML views.
It is **lightweight, fast, and powerful**, designed specifically for Laravel.

2. How do you pass data from controller to Blade?

1. Using `with()` method-

```
public function index() {
   return view('home')->with('name', 'Akash');
}
<!-- home.blade.php -->
<h1>Hello, {{ $name }}</h1>
```

2. Passing an array to `view()`

```
public function index() {
   return view('home', ['name' => 'Akash', 'skill' => 'Laravel']);
}
<h1>{{ $name }} knows {{ $skill }}</h1>
```

3. Passing with `compact()` –The compact() function creates an array from variables and their values.

```
public function index() {
   $name = "Akash";
   $skill = "PHP";
   return view('home', compact('name', 'skill'));
}
<p>{{ $name }} is skilled in {{ $skill }}</p>
```

4. Difference between `{{ $var }}` and `{!! $var !!}` in Blade.

`{{ $var }}` → **Safe, escaped output,** Prevents **XSS (Cross-Site Scripting) attacks**. (default, recommended).

`{!! $var !!}` → **Raw, unescaped output** (dangerous if user input is included).

5. How do you include one Blade file into another?

1. Using `@include` (most common)
2. Using `@includeIf`- Includes the file **only if it exists**.
3. @includeWhen - Includes file **only when a condition is true**
4. @includeUnless- includes only if condition is **false**.
5. @each - Renders a view for each item in a collection.

6. How do you use **Blade components**?
7. What is **@csrf** directive in Blade?

The `@csrf` directive is used inside **HTML forms** in Blade templates to generate a **CSRF (Cross-Site Request Forgery) token** automatically.
Laravel uses this token to **protect your application from CSRF attacks** by verifying that the request really came from your own site and not from a malicious external site.

8. How do you loop through data in Blade?

5. Security

1. What is **CSRF** protection in Laravel?
2. How does Laravel prevent **SQL Injection**?
3. How does Laravel handle **XSS protection**?
4. How do you implement authentication in Laravel?
5. What is **Laravel Sanctum** and **Laravel Passport**?
6. How to encrypt and decrypt data in Laravel?

Laravel Interview Question – 2

1. @csrf token kyo use karte hai laravel me form me ?

   Ans- Laravel me **@csrf token** form me isliye use kiya jata hai taaki **CSRF (Cross-Site Request Forgery) attacks** se application ko protect kiya ja sake.

- Jab bhi aap form submit karte ho, ye token form ke sath send hota hai.

- Server verify karta hai ki jo token aaya hai wo session me stored token se match karta hai ya nahi.

CSRF Token Kyo Zaruri Hai?

(a) Security ke liye
(b) **Forgery ko prevent karne ke liye:**
Agar token match nahi kare to Laravel automatically `419 Page Expired` error deta hai aur request ko reject kar deta hai.
(c) **User ke session ko protect karne ke liye:**
Sirf wahi request valid hoti hai jisme proper token ho, baaki sab reject ho jati hain.

2. custome middileware kaha per define karte hai ?

Ans- (1). **Middleware Create Karna**
php artisan make:middleware CheckUserRole

(2). **Middleware Logic Likho**

```
<?php

namespace App\Http\Middleware;
use Closure;
use Illuminate\Http\Request;
class CheckUserRole
{
   public function handle(Request $request, Closure $next)
   {
      if ($request->user() && $request->user()->role !== 'admin') {
         return redirect('/no-access');
      }
      return $next($request);
   }
}
```

3. Middleware Register Karna

Middleware ko Laravel me register karna zaruri hai. Ye do tarike se ho sakta hai:

(A) Global Middleware

- Sabhi requests par apply hoga.
- Register kare: `app/Http/Kernel.php` me `$middleware` array ke andar:

```
protected $middleware = [
   // ...
   \App\Http\Middleware\CheckUserRole::class,
```

(B) Route Middleware

- Sirf specific routes ke liye apply hoga.
- Register kare: `app/Http/Kernel.php` me `$routeMiddleware` array ke andar:

```
protected $routeMiddleware = [

   // ...

   'checkRole' => \App\Http\Middleware\CheckUserRole::class,
```

];

Phir route me use kare:

```
Route::get('/admin', function () {
    return 'Admin Panel';
})->middleware('checkRole');
```

3. observer kya hota hai laravel me?

Laravel me **Observer** ek aisa feature hai jo aapko model ke lifecycle events (jaise create, update, delete) ko listen karke un par automatically kuch actions perform karne ki facility deta hai.

Ye ek tarah ka **event listener** hota hai, lekin specifically **Eloquent models ke liye design kiya gaya hai**.

Observer Ka Use Kyu Hota Hai?

- Code ko **clean aur organized** rakhne ke liye
- Model ke lifecycle events ko handle karne ke liye (jaise email bhejna jab user create ho, log record karna jab product delete ho, etc.)
- Reusable aur centralized logic likhne ke liye

Model ke Events jo Observer Handle Kar Sakta Hai

- `creating` → Jab model create hone se pehle call hota hai
- `created` → Jab model successfully create ho jata hai
- `updating` → Jab model update hone se pehle call hota hai
- `updated` → Jab model successfully update ho jata hai
- `deleting` → Jab model delete hone se pehle call hota hai
- `deleted` → Jab model successfully delete ho jata hai
- `restoring` → Jab soft deleted model restore hone se pehle call hota hai
- `restored` → Jab soft deleted model successfully restore ho jata hai

Observer Kaise Banate Hai?

1. Observer Create Karna

php artisan make:observer UserObserver --model=User

2. Observer Me Methods Define Karna

```
namespace App\Observers;

use App\Models\User;

class UserObserver
{
    public function created(User $user)
    {
        // Jab naya user create ho
        \Log::info('New user created: ' . $user->email);
    }

    public function deleting(User $user)
    {
        // Jab user delete hone wala ho
        \Log::warning('User deleting: ' . $user->email);
    }}
```

3. Observer Ko Register Karna

Aap `App\Providers\EventServiceProvider.php` me register karte ho:

```
use App\Models\User;
use App\Observers\UserObserver;

public function boot(): void
{
    User::observe(UserObserver::class);
}
```

4. Jab Bhi User Create / Delete / Update Hoga

Aapka observer automatic run hoga bina har jagah code likhne ke.

Observer Ka Real-Life Example

- User create hone par **welcome email send karna**
- Product delete hone par **related records delete karna**
- Order create hone par **invoice generate karna**
- User update hone par **activity log record karna**

4. How to send mail?

(a) php artisan make:mail FormDataMail
(b) app/Mail/FormDataMail.php

```php
<?php

namespace App\Mail;

use Illuminate\Bus\Queueable;
use Illuminate\Contracts\Queue\ShouldQueue;
use Illuminate\Mail\Mailable;
use Illuminate\Mail\Mailables\Content;
use Illuminate\Mail\Mailables\Envelope;
use Illuminate\Queue\SerializesModels;

class FormDataMail extends Mailable
{
    use Queueable, SerializesModels;

    public $formData;

    /**
     * Create a new message instance.
     */
    public function __construct($formData)
    {
        $this->formData = $formData;
    }

    /**
     * Get the message envelope.
     */
    public function envelope(): Envelope
    {
        return new Envelope(
            subject: 'New Form Submission',
        );
    }

    /**
     * Get the message content definition.
     */
    public function content(): Content
    {
        return new Content(
            view: 'emails.form_data', // <-- yaha apna email view ka naam daale
            with: ['formData' => $this->formData]
        );
    }

    /**
     * Get the attachments for the message.
     *
     * @return array<int, \Illuminate\Mail\Mailables\Attachment>
```

```
     */
    public function attachments(): array
    {
        return [];
    }
}
```

}

(c) resources/views/emails/form_data.blade.php

```
<h2>New Form Submission</h2>
<p><strong>Name:</strong> {{ $formData['name'] }}</p>
<p><strong>Email:</strong> {{ $formData['email'] }}</p>
<p><strong>Gender:</strong> {{ $formData['gender'] }}</p>
<p><strong>Language:</strong> {{ implode(', ', $formData['language']) }}</p>
<p><strong>City:</strong> {{ $formData['city'] }}</p>
@if(!empty($formData['photo']))
<p><strong>Photo:</strong> {{ $formData['photo'] }}</p>
@endif
```

(d) Step 4: Controller Update Kare

```
use App\Mail\FormDataMail;
use Illuminate\Support\Facades\Mail;

public function store(Request $request){
   $validated = $request->validate([
      'name' => 'required|string',
      'email' => 'required|email',
      'gender' => 'required',
      'language' => 'required',
      'city' => 'required',
      'photo' => 'nullable'
   ]);

   $curd = new curd;
   $curd->name = $validated['name'];
   $curd->email = $validated['email'];
   $curd->gender = $validated['gender'];
   $curd->language = implode(',', $validated['language']);
   $curd->city = $validated['city'];

   if($request->hasFile('photo')){
      $photofile = $request->file('photo')->store('photos', 'public');
      $curd->photo = $photofile;
   }
```

```
    $curd->save();

    // Send email after saving data
    Mail::to('akashdhiman716@gmail.com')->send(new FormDataMail($validated));

    return redirect()->back()->with('success', 'Data saved and email sent successfully!');
}
```

(e) .env

```
APP_NAME=Laravel
APP_ENV=local
APP_KEY=base64:6aNYgQ1Mnn6SSHRPKs3aDXW7A5R3Ph3C5vvgqr7mug0=
APP_DEBUG=true
APP_URL=http://localhost:8000

APP_LOCALE=en
APP_FALLBACK_LOCALE=en
APP_FAKER_LOCALE=en_US

APP_MAINTENANCE_DRIVER=file
# APP_MAINTENANCE_STORE=database

PHP_CLI_SERVER_WORKERS=4

BCRYPT_ROUNDS=12

LOG_CHANNEL=stack
LOG_STACK=single
LOG_DEPRECATIONS_CHANNEL=null
LOG_LEVEL=debug

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=mig7
DB_USERNAME=root
DB_PASSWORD=

SESSION_DRIVER=database
SESSION_LIFETIME=120
SESSION_ENCRYPT=false
SESSION_PATH=/
SESSION_DOMAIN=null

BROADCAST_CONNECTION=log
FILESYSTEM_DISK=local
QUEUE_CONNECTION=database

CACHE_STORE=database
# CACHE_PREFIX=

```

```
MEMCACHED_HOST=127.0.0.1

REDIS_CLIENT=phpredis
REDIS_HOST=127.0.0.1
REDIS_PASSWORD=null
REDIS_PORT=6379

MAIL_MAILER=smtp
MAIL_HOST=smtp.gmail.com
MAIL_PORT=587
MAIL_USERNAME=akashdhiman716@gmail.com
MAIL_PASSWORD=cgsnwugorfyvlbqm
MAIL_ENCRYPTION=tls
MAIL_FROM_ADDRESS="akashdhiman716@gmail.com"
MAIL_FROM_NAME="Laravel Form"

AWS_ACCESS_KEY_ID=
AWS_SECRET_ACCESS_KEY=
AWS_DEFAULT_REGION=us-east-1
AWS_BUCKET=
AWS_USE_PATH_STYLE_ENDPOINT=false

VITE_APP_NAME="${APP_NAME}"
```

Q-1 Laravel me **Authentication** aur **Authorization** do alag concepts hai, lekin dono connected hote hai.

1. Authentication

Kya hai?
Authentication ka matlab hai check karna ki **user kaun hai?**
Kyu zaroori hai?
Taaki system ko pata chale ki login karne wala user genuine hai ya nahi.

2. Authorization

Kya hai?
Authorization ka matlab hai decide karna ki **user kya kar sakta hai aur kya nahi?**
Kyu zaroori hai?
Taaki har user ko sirf wahi access mile jo uske role ya permission ke hisab se ho.

Q-2 Factory and seeder ?

1. Seeder Kya Hai?

Seeder ka kaam hai database me **predefined ya fixed data** insert karna.
Iska use aap tab karte ho jab aapko kuch specific default data chahiye ho.

Example:
Default **admin user** create karna.
Kuch **static categories**, jaise "Science, Commerce, Arts" insert karna.
Application ke liye **basic setup data** dalna.

2. Factory Kya Hai?

Factory ka kaam hai **random ya fake data generate karna**, mostly testing aur development ke liye.
Factory ke sath aap thousands of records easily generate kar sakte ho.
Example:
100 students ke **random names, emails, roll numbers** banana.
E-commerce site me 500 fake products generate karna.

Feature	Seeder	Factory
Data Type	Fixed/Predefined	Random/Fake
Use Case	Default setup, admin user, roles	Testing, Development, Bulk data
How to Define?	Manually array ke through	Faker ke through rules define karte ho
Example Record	Ek fixed admin user	100 random students

Aap Laravel me **Factory + Seeder** ka use karke apne `curds` table me dummy data dal sakte ho.
Main aapko step by step process bata raha hoon:

(1) Model Check

Aapke paas model Curd hai aur table ka naam curds hai. Ye sahi hai.

(2) Factory Banana

php artisan make:factory CurdFactory --model=Curd
Ye `database/factories/CurdFactory.php` file banayega.

3. Factory File Edit Karna
`CurdFactory.php` file me aise likhen:

```
<?php
```

```
namespace Database\Factories;
use Illuminate\Database\Eloquent\Factories\Factory;
use App\Models\Curd;
class CurdFactory extends Factory
{
   protected $model = Curd::class;
   public function definition(): array
   {
      return [
         'name' => $this->faker->name(),
         'email' => $this->faker->unique()->safeEmail(),
         'gender' => $this->faker->randomElement(['Male', 'Female', 'Other']),
         'language' => $this->faker->randomElement(['English', 'Hindi', 'Punjabi']),
         'city' => $this->faker->city(),
         'photo' => $this->faker->imageUrl(200, 200, 'people', true, 'Profile')
      ];
   }
}
```

4. Seeder Banana

php artisan make:seeder CurdSeeder

Ye `database/seeders/CurdSeeder.php` file banayega

5. Seeder File Edit Karna

```
<?php
namespace Database\Seeders;
use Illuminate\Database\Seeder;
use App\Models\Curd;
class CurdSeeder extends Seeder
{
   public function run(): void
   {
      Curd::factory()->count(10)->create(); // 10 records create karega
   }
}
```

6. Seeder Ko Register Karna

`database/seeders/DatabaseSeeder.php` file me `run()` ke andar add kare:

```
$this->call([
   CurdSeeder::class,
]);
```

7. Seeder Run Karna

php artisan db:seed